

Best Practices Guide for Angular / React Full Stack Capstone Project Implementation

Version 1.0

Applicable For:

- Angular Full Stack Projects
 - React Full Stack Projects
 - Spring Boot / .NET Backend Projects
 - JFS / FSD Capstone Assessments
 - Client SME Reviews
 - Placement Readiness Evaluations
-

1. Project Planning & Business Understanding

1.1 Understand the Business Problem

Before writing code, teams must clearly identify:

Problem Statement

What business challenge is being solved?

Stakeholders

Who will use the application?

Examples:

- Customer
- Administrator
- Manager
- Auditor

Expected Outcomes

- Process automation
 - Reduced manual effort
 - Improved reporting
 - Better user experience
 - Secure data management
-

1.2 User Story Creation

Example:

"As a customer, I should be able to register and log in securely."

"As an administrator, I should be able to manage users and generate reports."

Every feature implemented must map to a user story.

2. Solution Architecture Guidelines

2.1 Follow a 3-Tier Architecture

Presentation Layer

Angular / React

Responsibilities:

- UI rendering
- Form validation
- Navigation
- API consumption

Business Layer

Spring Boot / ASP.NET Core

Responsibilities:

- Business rules
- Authentication

- Authorization
- API management

Data Layer

SQL Server / MySQL / PostgreSQL

Responsibilities:

- Data storage
 - Data integrity
 - Relationships
 - Transactions
-

2.2 Architecture Artifacts

Every project must include:

High-Level Architecture Diagram

Include:

Client Application

↓

REST APIs

↓

Business Services

↓

Database

Sequence Diagram

Show:

User Action

↓

API Request

↓

Business Processing

↓

Database Interaction

↓

Response

3. Frontend Development Best Practices

3.1 Component Design

Angular

Use:

- Feature Modules
- Shared Modules
- Reusable Components

Avoid:

- Large Components (>500 lines)
- Business Logic inside Templates

React

Use:

- Functional Components
- Hooks
- Reusable UI Components

Avoid:

- Deep Prop Drilling
 - Excessive State Duplication
-

3.2 State Management

Recommended:

Angular:

- Services
- RxJS
- Signals (Latest Angular)

React:

- Context API
- Redux Toolkit
- Zustand

State should be centralized and predictable.

3.3 Forms & Validation

Mandatory:

- ✓ Required Validation
- ✓ Email Validation
- ✓ Password Validation
- ✓ Length Validation
- ✓ Custom Business Rules

Never rely only on frontend validation.

3.4 Routing & Navigation

Implement:

- ✓ Protected Routes
 - ✓ Role-Based Navigation
 - ✓ Unauthorized Access Handling
 - ✓ Route Guards
-

4. Backend Development Best Practices

4.1 Layered Architecture

Controllers

↓

Services

↓

Repositories

↓

Database

Business logic should NEVER exist inside controllers.

4.2 API Design Standards

Use REST conventions:

1. GET → Retrieve
2. POST → Create
3. PUT → Update
4. DELETE → Remove

5. PATCH → Partial Update

4.3 DTO Implementation

Use DTOs to:

- Hide internal database structures
- Improve security
- Reduce payload size

Avoid returning entity objects directly.

4.4 Dependency Injection

Use constructor injection.

Avoid manual object creation using "new" wherever possible.

5. Security & Authentication Standards

5.1 JWT Authentication

Must include:

- ✓ Login
 - ✓ Token Generation
 - ✓ Token Validation
 - ✓ Expiry Management
 - ✓ Logout Handling
-

5.2 Password Security

Use:

- BCrypt
- PBKDF2
- ASP.NET Identity Hashing

Never store plaintext passwords.

5.3 Authorization

Implement Role-Based Access Control (RBAC)

Example:

- Admin
- Manager
- Customer
- Auditor

Each role should have separate permissions.

6. Database Design Best Practices

6.1 Database Design

Follow:

- ✓ First Normal Form
 - ✓ Second Normal Form
 - ✓ Third Normal Form
-

6.2 Relationships

Implement:

- Primary Keys
- Foreign Keys
- Constraints

Document all relationships using ER Diagrams.

6.3 Query Optimization

Use:

1. Indexes
2. Pagination
3. Filtered Queries
4. Parameterized Queries

Avoid:

- SELECT *
 - Hardcoded SQL
 - Unoptimized Joins
-

7. Business Logic Implementation

7.1 Service Layer Standards

All business rules must be implemented in:

1. Services
2. Managers
3. Domain Services

Examples:

1. Order Validation
2. Payment Validation
3. Inventory Verification
4. Workflow Approvals

7.2 Workflow Management

Each workflow should support:

- Validation
 - Processing
 - Audit Tracking
 - Error Recovery
-

8. API Integration Standards

Frontend must:

- ✓ Consume APIs through Services
- ✓ Handle Loading States
- ✓ Handle API Errors
- ✓ Manage Authentication Tokens
- ✓ Support Retry Mechanisms

Avoid calling APIs directly inside components.

9. Exception Handling & Logging

Backend

Implement:

- Global Exception Handler
- Standard Error Response
- Logging Framework
- Examples:

- Log4j
 - SLF4J
 - Serilog
 - NLog
-

Frontend

Display:

- Friendly Messages
 - Validation Errors
 - Network Errors
 - Session Expiry Alerts
-

10. Testing & Quality Assurance

Minimum Testing Coverage

- **Functional Testing**
 - CRUD Operations
 - Authentication
 - Business Flows
 - **API Testing**
 - Postman
 - Swagger
 - Bruno
 - **Edge Cases**
 - Null Inputs
 - Invalid Inputs
 - Large Datasets
 - Unauthorized Access
-

11. Documentation Standards

Every Project Must Include

Business Documentation

1. Problem Statement
 2. Objectives
 3. User Stories
 4. Use Cases
 5. Scope
-

Technical Documentation

1. Architecture Diagram
 2. Database Design
 3. API Documentation
 4. Deployment Guide
 5. Folder Structure
-

Testing Documentation

1. Test Cases
 2. Execution Results
 3. Defect Summary
-

12. Naming Conventions

Angular / React

1. Components
 2. UserListComponent
 3. LoginPage
 4. DashboardCard
-

APIs

1. /api/users
 2. /api/orders
 3. /api/reports
-

Database Tables

1. users
 2. orders
 3. payments
 4. audit_logs
-

Services

1. UserService
 2. OrderService
 3. AuthService
-

13. Plagiarism Prevention Guidelines

Mandatory Requirements

- ✓ Maintain Git Repository History
- ✓ Use Original Business Logic
- ✓ Create Custom UI Designs
- ✓ Modify Sample Datasets
- ✓ Cite External References
- ✓ Generate Original Documentation

Avoid:

- ✗ Copy-Pasting GitHub Projects
 - ✗ Using Generated Projects Without Customization
 - ✗ Reusing Public Capstones Without Modification
-

14. Deployment & Submission Guidelines

Submission Package Must Include:

Engineering Pack

1. Source Code
 2. Database Scripts
 3. Environment Files
 4. API Collections
-

Documentation Pack

1. Project Report
 2. Architecture Diagrams
 3. ER Diagrams
 4. User Manual
 5. Deployment Guide
-

Validation Pack

1. Postman Collection
 2. Testing Evidence
 3. Screenshots
 4. Execution Logs
-

15. SME Walkthrough Checklist

Task	Status
Business Problem Defined	☐
User Stories Created	☐
Architecture Diagram Prepared	☐
Database Designed & Normalized	☐
Angular/React UI Completed	☐
APIs Developed	☐
JWT Security Implemented	☐
CRUD Operations Working	☐
Exception Handling Added	☐
Testing Completed	☐
Documentation Prepared	☐
Deployment Verified	☐
Final Presentation Ready	☐

Final Principle

A Full Stack Project is not evaluated solely on functionality.

It is evaluated on:

- Business Understanding
- Architecture Quality
- Security
- Database Design
- Coding Standards
- Testing
- Documentation
- Scalability

- Technical Defense

A successful capstone project should be functional, secure, maintainable, scalable, and professionally documented.